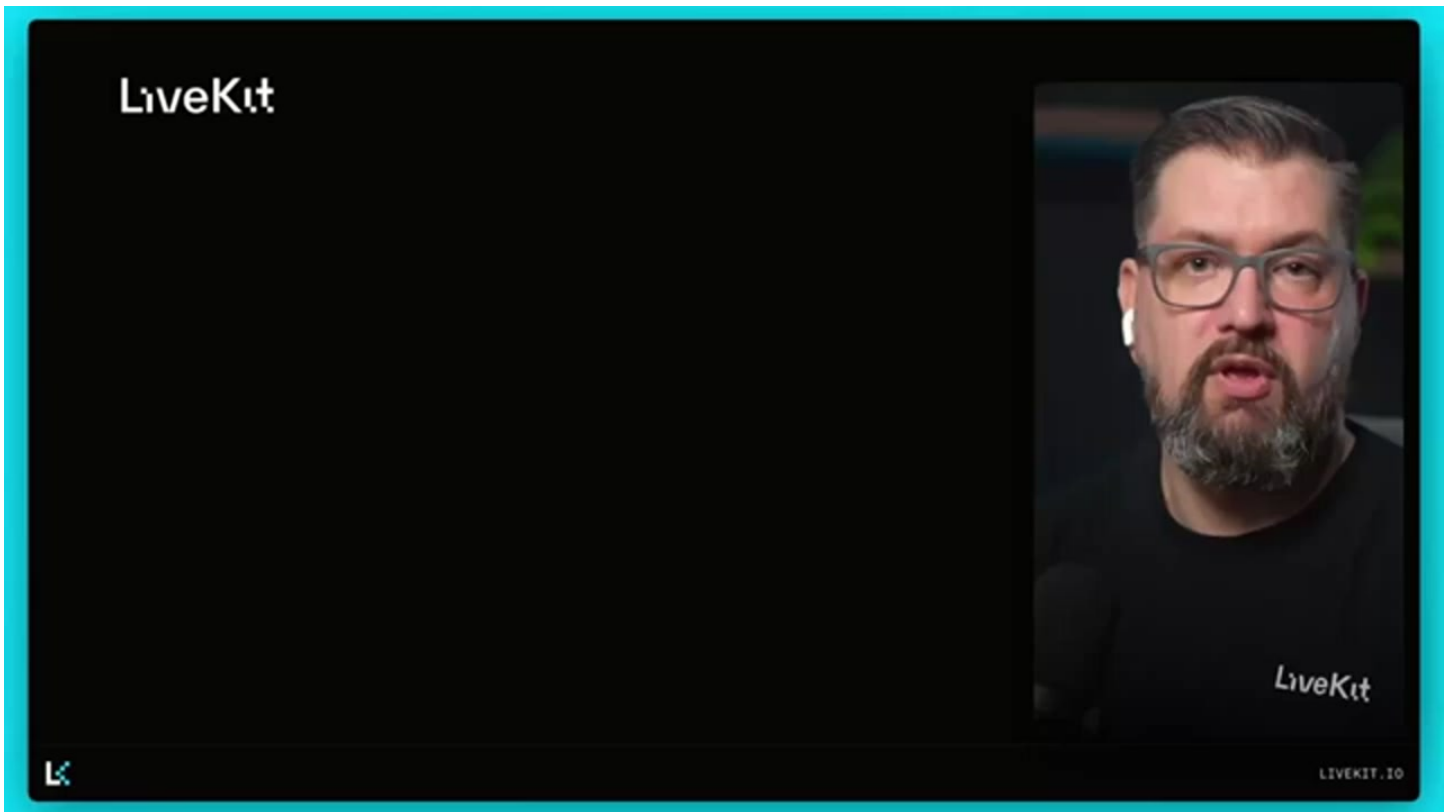


Lesson 0: Building Production-Ready Voice Agents

Section: LiveKit Voice AI Course

Summary: Coverage of Lesson 0: Building Production-Ready Voice Agents (00:00 - 03:03)

■ [00:00 - 00:20]



You've probably talked to a voice AI recently and maybe it was frustrating. Maybe it cut you off. Maybe it felt robotic. Here's the thing. Building a voice agent that actually feels natural is harder than it looks. But once you understand the fundamentals, you can build something that people genuinely want to talk to, don't mind talking to,

■ [00:20 - 00:40]

LiveKit

Build production-ready
voice agents

Hands-on workshop



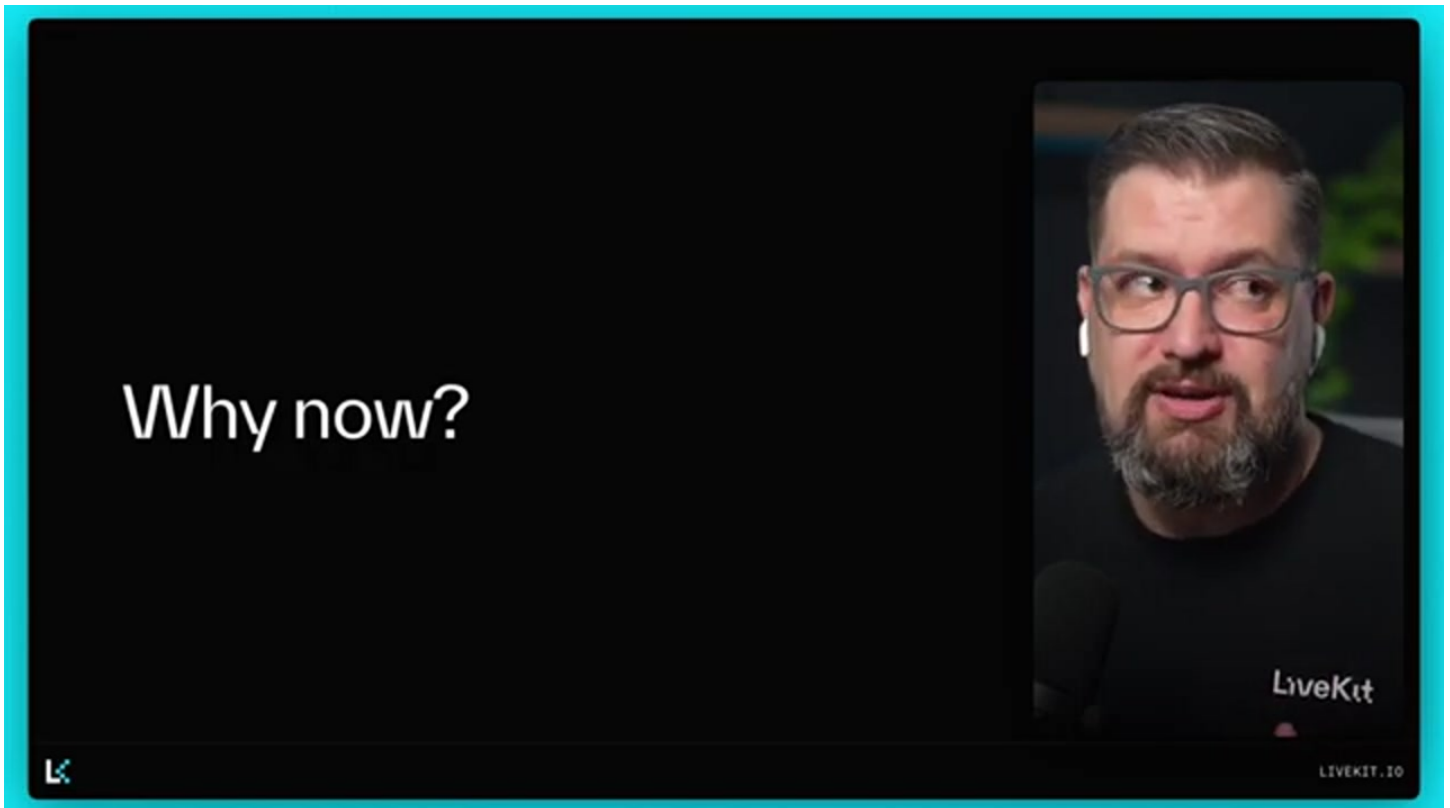
LiveKit



LIVEKIT.IO

want to talk to, don't mind talking to, and actually prefer to talk to. And that's what this workshop is about. You don't need prior experience with voice AI, machine learning, or real-time systems. I'll explain all of those concepts as we go. Voice AI hit an inflection point. Speechtoext models are faster and more accurate. LLMs can handle nuanced conversations. Text to

■ [00:40 - 01:00]



handle nuanced conversations. Text to speech sounds human. These pieces exist. The challenge is wiring them together into something that feels responsive and natural. Now, a year ago, building this required specialized engineering knowledge and months of hard work. Today, you can get a working voice agent running in just a few minutes. So, here's what you're going to learn throughout this workshop. First, how the

■ [01:00 - 01:20]

What you'll learn



LIVEKIT.IO

throughout this workshop. First, how the voice pipeline works. That's VAD, ST, LLM, and TTS. And then, why latency matters and how to uh minimize it. You'll learn techniques for natural turntaking so your agent doesn't interrupt you mid-sentence. You'll add personality to your agent and set up provider failovers uh so that it stays

What you'll learn

- Voice pipeline fundamentals
- Latency optimization
- Natural turn-taking
- Personality & failover



LIVEKIT.IO

provider failovers uh so that it stays online even when a service goes down. And later you'll integrate external tools and APIs. And by the end you'll build real workflows with handoffs and consent collection. And the result is that you'll have an agent that listens, thinks, and responds in real time, not a toy demo. A foundation that you can actually ship. Each lesson adds a new

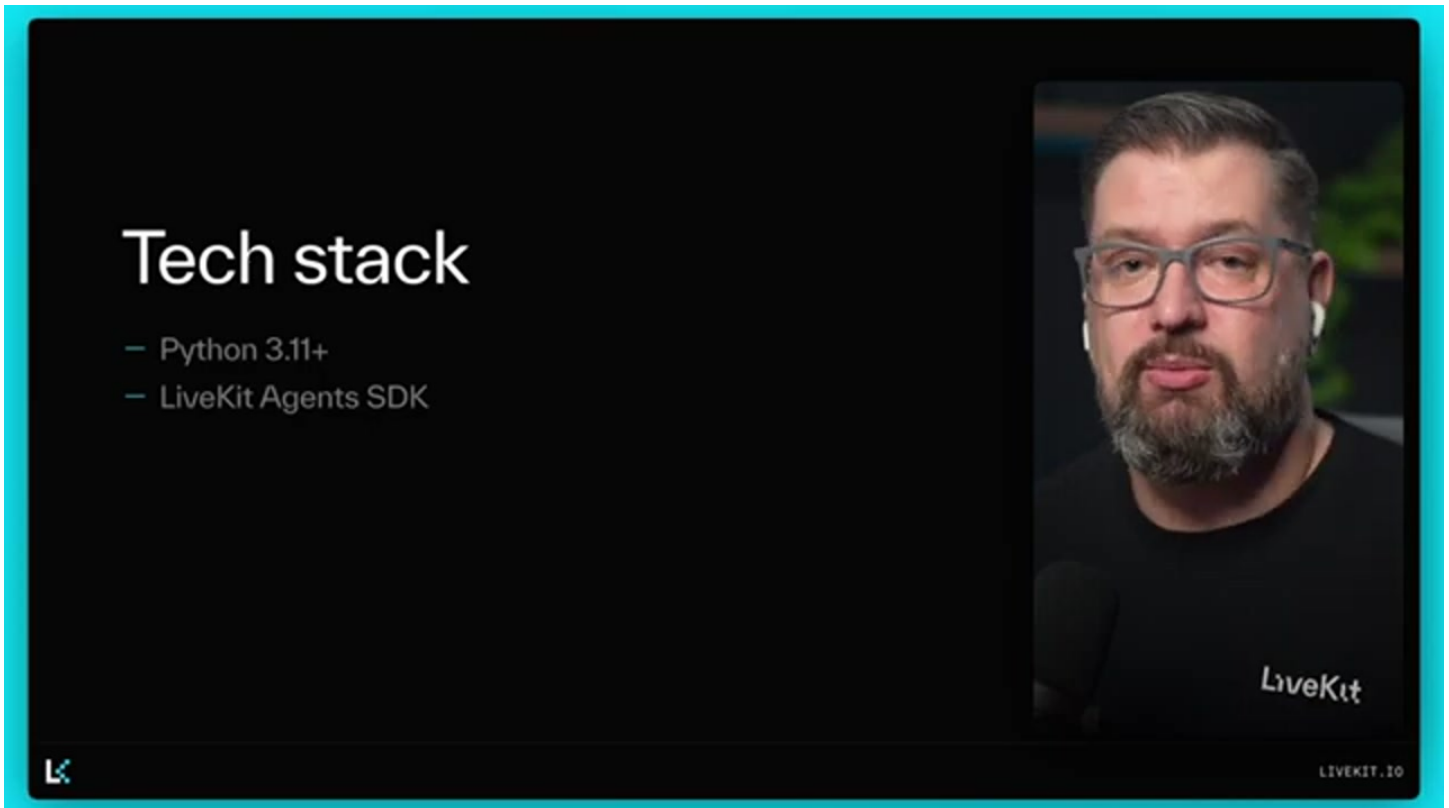
What you'll learn

- Voice pipeline fundamentals
- Latency optimization
- Natural turn-taking
- Personality & failover
- Tool integration
- Workflows & handoffs



LIVEKIT.IO

actually ship. Each lesson adds a new capability, and each one builds on the last. You'll use the LifeKit agents SDK for Python. LifeKit handles the real-time infrastructure so that you can focus on the uh agentic logic. But note that while this workshop uses Python, LifeKit also offers a TypeScript agents



The video frame shows a presentation slide with a black background and a cyan border. On the left, the title 'Tech stack' is displayed in a large, white, sans-serif font. Below the title, there is a list of two items: 'Python 3.11+' and 'LiveKit Agents SDK', both in a smaller, white, sans-serif font. On the right side of the slide, there is a portrait of a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. The man is looking directly at the camera. In the bottom left corner of the slide, there is a small white logo consisting of a stylized 'L' and 'K'. In the bottom right corner, the text 'LIVEKIT.IO' is visible in a small, white, sans-serif font.

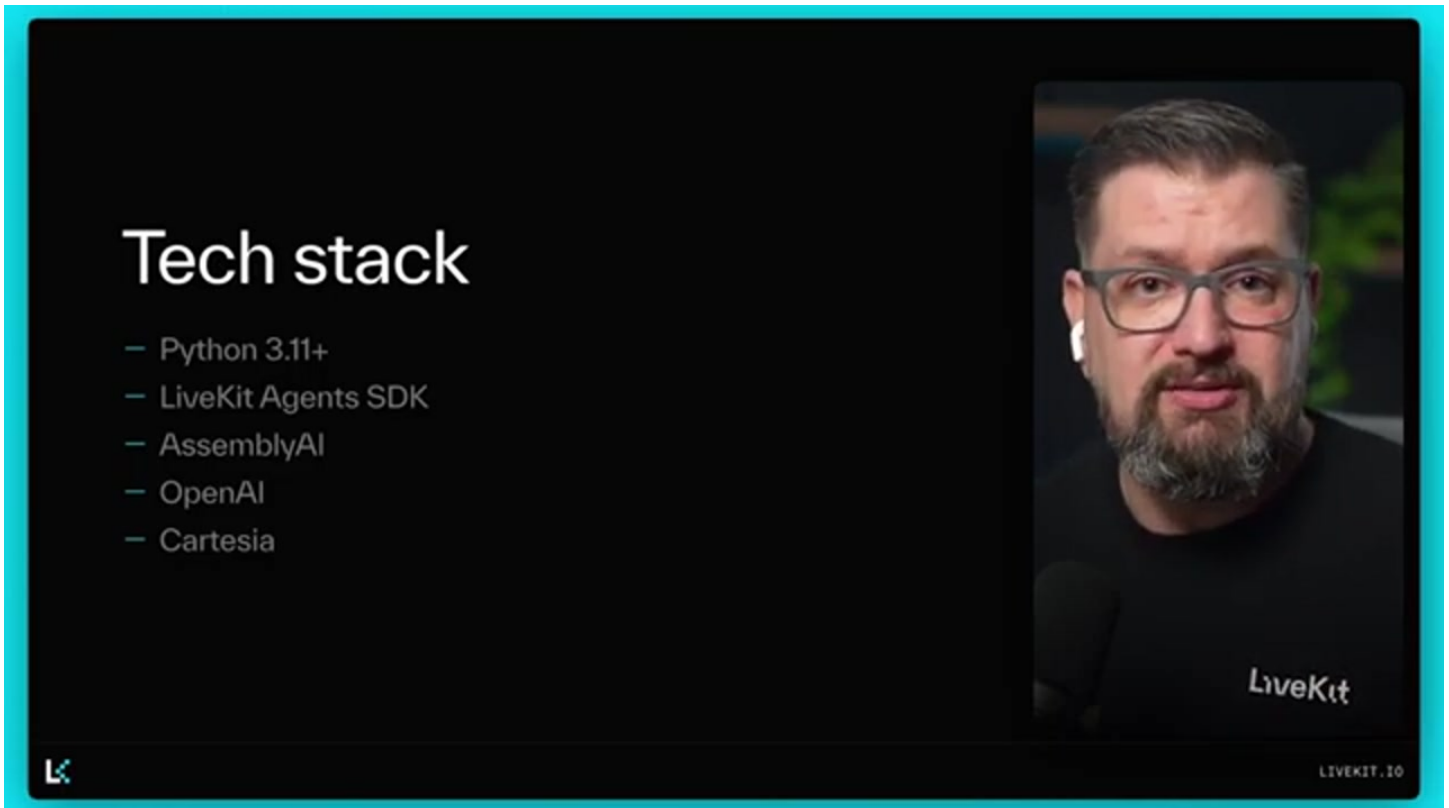
Tech stack

- Python 3.11+
- LiveKit Agents SDK

LiveKit

LIVEKIT.IO

LifeKit also offers a TypeScript agents SDK if you prefer working in JavaScript or TypeScript. And these are the models that we'll use throughout the workshop. Uh but they're not the only options. The agents SDK supports a wide range of providers for each component. If you want to use Azure for ST, Anthropic for your LLM, and 11 Labs for your TTS, you



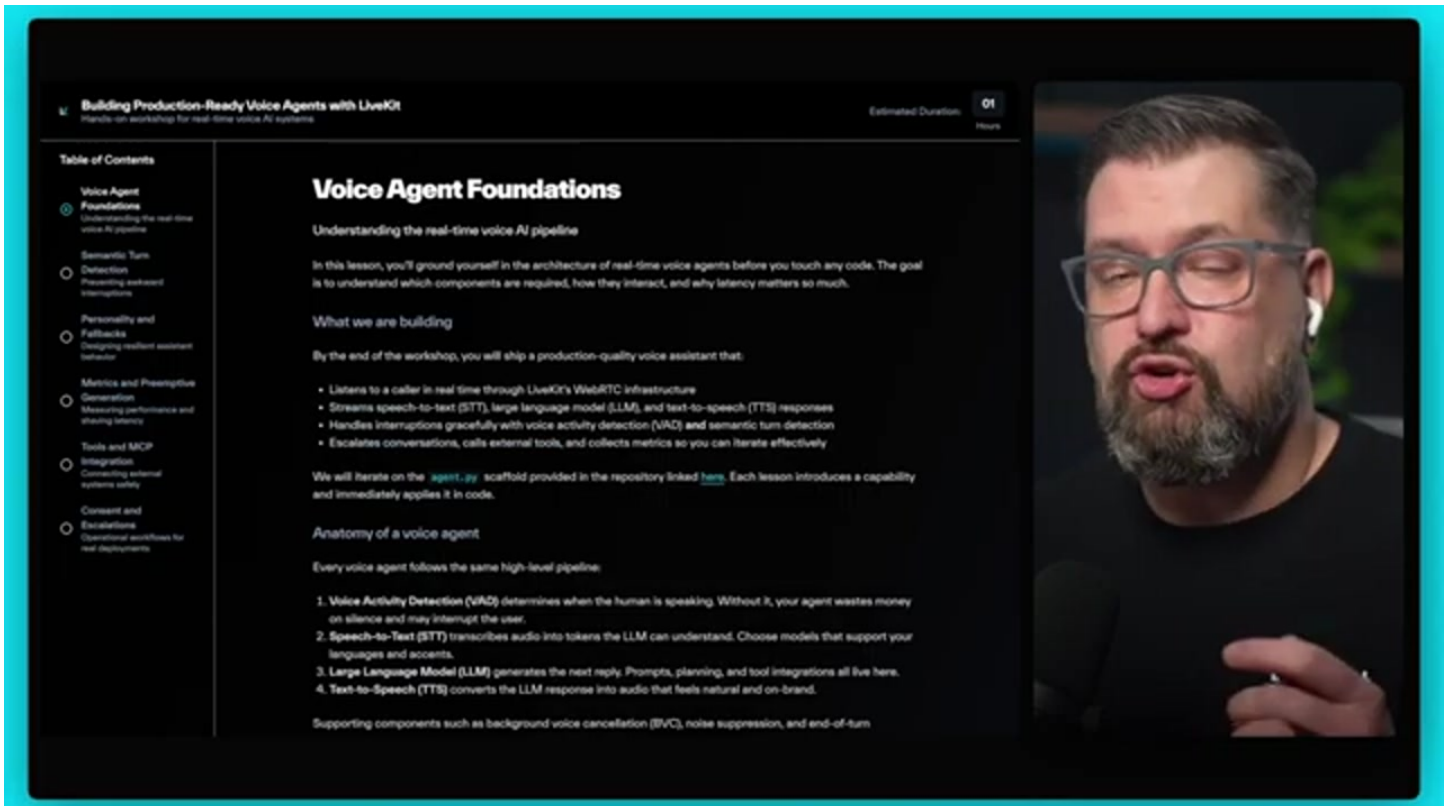
The slide features a dark background with a cyan border. On the left, the title 'Tech stack' is displayed in a large, white, sans-serif font. Below the title is a bulleted list of technologies: Python 3.11+, LiveKit Agents SDK, AssemblyAI, OpenAI, and Cartesia. On the right side, there is a video inset showing a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. A microphone is positioned in front of him. In the bottom left corner of the slide is a small 'LK' logo, and in the bottom right corner is the text 'LIVEKIT.IO'.

Tech stack

- Python 3.11+
- LiveKit Agents SDK
- AssemblyAI
- OpenAI
- Cartesia

LK LIVEKIT.IO

your LLM, and 11 Labs for your TTS, you can swap them out with just a few lines of code. The patterns that you're going to learn will apply regardless of which providers you choose. And we'll use LiveKit inference, which comes with credits, so you'll get access to all of these models so that you can follow along without spending any money. There's also uh written materials that go along with each video lesson, so you



The screenshot shows a video player interface. On the left is a 'Table of Contents' sidebar with a list of topics: Voice Agent Foundations (selected), Semantic Turn, Detection, Personality and Feedbacks, Metrics and Proactive, Generation, Tools and MCP, Integration, Consent and Escalations. The main content area displays the 'Voice Agent Foundations' slide. The slide title is 'Voice Agent Foundations'. Below it is the subtitle 'Understanding the real-time voice AI pipeline'. The main text explains the goal of the lesson: to ground oneself in the architecture of real-time voice agents before touching any code. It lists what will be built by the end of the workshop: a production-quality voice assistant that listens to a caller in real time through LiveKit's WebRTC infrastructure, streams speech-to-text (STT), large language model (LLM), and text-to-speech (TTS) responses, handles interruptions gracefully with voice activity detection (VAD) and semantic turn detection, and escalates conversations, calls external tools, and collects metrics. It mentions iterating on the 'agent.py' scaffold provided in the repository linked 'here'. The slide also includes an 'Anatomy of a voice agent' section, stating that every voice agent follows the same high-level pipeline: 1. Voice Activity Detection (VAD) determines when the human is speaking. Without it, your agent wastes money on silence and may interrupt the user. 2. Speech-to-Text (STT) transcribes audio into tokens the LLM can understand. Choose models that support your languages and accents. 3. Large Language Model (LLM) generates the next reply. Prompts, planning, and tool integrations all live here. 4. Text-to-Speech (TTS) converts the LLM response into audio that feels natural and on-brand. Supporting components such as background voice cancellation (BVC), noise suppression, and end-of-turn are also mentioned. On the right side of the video player, there is a video feed of a man with a beard and glasses, wearing a black shirt, speaking into a microphone.

Building Production-Ready Voice Agents with LiveKit
Hands-on workshop for real-time voice AI systems

Estimated Duration: 01 Hours

Table of Contents

- Voice Agent Foundations (selected)
- Semantic Turn
- Detection
- Personality and Feedbacks
- Metrics and Proactive
- Generation
- Tools and MCP
- Integration
- Consent and Escalations

Voice Agent Foundations

Understanding the real-time voice AI pipeline

In this lesson, you'll ground yourself in the architecture of real-time voice agents before you touch any code. The goal is to understand which components are required, how they interact, and why latency matters so much.

What we are building

By the end of the workshop, you will ship a production-quality voice assistant that:

- Listens to a caller in real time through LiveKit's WebRTC infrastructure
- Streams speech-to-text (STT), large language model (LLM), and text-to-speech (TTS) responses
- Handles interruptions gracefully with voice activity detection (VAD) and semantic turn detection
- Escalates conversations, calls external tools, and collects metrics so you can iterate effectively

We will iterate on the `agent.py` scaffold provided in the repository linked [here](#). Each lesson introduces a capability and immediately applies it in code.

Anatomy of a voice agent

Every voice agent follows the same high-level pipeline:

1. **Voice Activity Detection (VAD)** determines when the human is speaking. Without it, your agent wastes money on silence and may interrupt the user.
2. **Speech-to-Text (STT)** transcribes audio into tokens the LLM can understand. Choose models that support your languages and accents.
3. **Large Language Model (LLM)** generates the next reply. Prompts, planning, and tool integrations all live here.
4. **Text-to-Speech (TTS)** converts the LLM response into audio that feels natural and on-brand.

Supporting components such as background voice cancellation (BVC), noise suppression, and end-of-turn

can use them as a reference while you code. And there's a link in the video description for that. Now, before you start the workshop, make sure that you have Python 3.11 or higher, the UV package manager, a code editor, a free LiveKit Cloud account, and the written guide open if you want to copy and paste code snippets. Again, there's links in the video description for all of these

What you'll need

- Python 3.11+
- uv package manager
- Code editor
- LiveKit Cloud account (free)
- Written guide



the video description for all of these materials. So, got all that? Let's start